Day 5: Observability and Operational Management

---

1. Cloud Monitoring and Logging in GCP

Definition

Observability = Ability to understand system behaviour using metrics, logs, and traces.

Cloud Monitoring and Logging in GCP help track performance, detect issues, and troubleshoot systems in real time.

Cloud Monitoring

Definition

Service to collect and visualize metrics such as CPU, memory, latency, and uptime.

Key Features

- Dashboards
- Metrics explorer
- Alerting policies
- Uptime checks

Cloud Logging

Definition

Service to collect, store, and analyze logs generated by applications and infrastructure.

Types of Logs

- Application logs
- System logs
- Audit logs

Scenario

Application slowdown

Problem:

- Users report latency issues

Solution:

- Use Monitoring to check CPU/memory
- Use Logging to inspect errors

Result:

- Root cause identified quickly

Case Study

E-commerce Platform

- Uses monitoring dashboards
- Uses logging for error tracking

Result:

- Faster issue resolution
- Improved user experience

---

2. GKE Dashboards and Log-Based Metrics

GKE Dashboards

Definition

Pre-built dashboards to monitor Kubernetes clusters, workloads, and resources.

Metrics Available

- Pod CPU and memory usage
- Node utilization
- Network traffic
- Pod restarts

Log-Based Metrics

Definition

Custom metrics created from log data for alerting and analysis.

Example

Log entry:

ERROR: Payment service failed

Create metric:

- Count number of error logs

Scenario

Frequent application errors

Problem:

- Hard to track manually

Solution:

- Create log-based metric

Result:

- Automated tracking and alerts

Case Study

Payment Gateway

- Created metric for failed transactions

Result:

- Immediate alerts when failures spike

---

3. Horizontal Pod Autoscaler (HPA)

Definition

HPA automatically scales the number of Pods based on CPU, memory, or custom metrics.

Example

```
kubectl autoscale deployment myapp-deployment \
  --cpu-percent=50 \
```

```
  --min=2 \
  --max=10
```

How It Works

- Monitors metrics (CPU/memory)
- Increases replicas during high load
- Decreases replicas during low load

Scenario

Traffic spike during sale

Problem:

- Application becomes slow

Solution:

- Configure HPA

Result:

- Pods scale automatically

Case Study

Streaming Application

- Uses HPA for scaling

Result:

- Handles peak traffic efficiently
- Optimizes cost during low usage

---

4. Debugging Failed Pods and Troubleshooting Deployments

Common Issues

Issue	Description
CrashLoopBackOff	Container keeps restarting
ImagePullBackOff	Image not found or access denied
Pending	Insufficient resources
OOMKilled	Out of memory

Debugging Commands

Check Pods:

kubectl get pods

Describe Pod:

kubectl describe pod <pod-name>

Check Logs:

kubectl logs <pod-name>

Exec into Pod:

kubectl exec -it <pod-name> -- /bin/bash

Scenario

Pod not starting

Steps:

1. Check pod status
2. Describe pod
3. Check logs
4. Fix issue

Case Study

Production Deployment Failure

Problem:

- Pods in CrashLoopBackOff

Root Cause:

- Wrong environment variable

Solution:

- Correct configuration

Result:

- Successful deployment

---

5. Hands-On: Configure HPA and Log-Based Alerts

Step 1: Deploy Application

```
kubectl apply -f deployment.yaml
```

Step 2: Enable Metrics Server

```
kubectl top pods
```

Step 3: Configure HPA

```
kubectl autoscale deployment myapp-deployment \
  --cpu-percent=50 \
  --min=2 \
  --max=5
```

Step 4: Verify HPA

```
kubectl get hpa
```

Step 5: Generate Load

- Use load testing tools or curl loops

Step 6: Create Log-Based Metric (GCP Console)

Steps:

1. Go to Logging
2. Create log-based metric
3. Define filter (e.g., ERROR)

Step 7: Create Alert Policy

- Link metric to alert
- Configure notification (email/SMS)

End-to-End Case Study (Day 5 Summary)

Scenario: High Traffic Web Application

Problem:

- Sudden traffic spikes
- Application crashes
- No visibility into issues

Solution

1. Enable monitoring and logging
2. Create dashboards
3. Configure HPA
4. Set up log-based alerts
5. Debug failures using kubectl

Result

- Real-time visibility
- Auto-scaling capability
- Faster troubleshooting
- Improved system reliability